

WIRELESS VEHICLE TELEMETRY & CONTROL SYSTEM

Engineering Notebook

Core Algorithm Design Entries: RPM Sensing, Attitude Estimation, Speed Control, Network Recovery

Sai Palepu

B.S. Mechanical Engineering, University of Georgia

companion to: WVTCS Algorithm Validation Report

Scope of This Notebook

This is a from-scratch rebuild of the four core algorithm entries from the original WVTCS engineering notebook — the hall-effect RPM measurement, the complementary filter, the PI speed controller, and the WiFi/MQTT reconnection logic — reorganized into a consistent entry format with clear objectives, pseudocode, and bench data. The content matches what the notebook originally established; only the presentation has changed.

Each entry here was later independently stress-tested against a much wider range of synthetic conditions than a single bench session covers, in a separate validation pass that found two real bugs. See the WVTCS Algorithm Validation Report ([wvtcs-validation repository](#)) for that analysis — the entries below reflect the algorithms as originally designed and bench-verified, before that later validation work.

Entry 04 — RPM Measurement — Hall-Effect Sensor

Objective

Measure wheel speed from a single-magnet hall-effect sensor. A fixed pulse-counting window is a poor fit for a single-magnet, low-RPM sensor, so the measurement is instead derived from the time between successive pulses.

Design

RPM is computed from the pulse period rather than a pulse count. A stale-timeout guards against reporting a stuck value forever: if no new pulse arrives within 1.5 seconds, the system assumes the wheel has stopped and reports 0 RPM.

```
on pulse_detected():
    dt = now() - last_pulse_time
    rpm = 60 / dt
    last_pulse_time = now()

on timer_tick():
    if now() - last_pulse_time > STALE_TIMEOUT:    # 1.5 s
        rpm = 0
```

Bench Verification

Verified against live pulses from the hall-effect sensor across the motor's operating range (see the duty-cycle/RPM table under Entry 08); RPM tracked as expected during normal operation.

Status: **Working as designed — bench-verified**

Entry 03 — Attitude Estimation — Complementary Filter

Objective

Produce a stable tilt/attitude estimate from an MPU-6050 by fusing the gyroscope's fast short-term response with the accelerometer's drift-free, gravity-referenced long-term reference.

Design

A complementary filter blends a gyro-integrated angle with an accelerometer-derived angle. The blend constant alpha weights how much the estimate trusts the gyro (fast, but drifts) versus the accelerometer (noisy, but stable long-term).

```
angle_gyro = angle + gyro_rate * dt
angle_accel = atan2(accel_y, accel_z) # gravity-referenced
angle = ALPHA * angle_gyro + (1 - ALPHA) * angle_accel
# ALPHA = 0.98
```

Bench Verification

Alpha was tuned empirically against a handful of candidate values on the bench; alpha = 0.98 was selected as the best balance of gyro responsiveness against accelerometer-corrected drift.

Status: **Working** — alpha = 0.98 selected

Entry 08 — Speed Control — PI Controller

Objective

Close the loop from a commanded RPM setpoint to a PWM duty output, so the wheel tracks a requested speed.

Design

A standard PI controller with a deadband floor (the motor doesn't turn below roughly 28% duty) and a saturation ceiling at 100% duty, with conditional anti-windup so the integral term stops accumulating once the output is already saturated.

```
error = setpoint - measured_rpm
integral += error * dt # frozen while output is saturated (anti-windup)
output = Kp * error + Ki * integral
output = clamp(output, 0.28, 1.0) # deadband floor / saturation ceiling

# tuned gains: Kp = 0.004 Ki = 0.003
```

Bench Data — Manual PWM Duty vs. Measured RPM

Commanded Duty	Measured RPM
30%	31 RPM
50%	54 RPM
75%	79 RPM
100%	97 RPM

Bench Verification

Closed-loop step response tested at a 100 RPM setpoint: settling time 2.2 seconds.

Status: **Working** — verified at 100 RPM step

Entry 06 — Network Recovery — WiFi / MQTT Reconnection

Objective

Automatically recover the ESP32's WiFi and MQTT broker connection after a drop, without hammering the broker on every retry.

Design

A connection state machine retries with exponential backoff between attempts, capped at 30 seconds.

```
delay = min(30 seconds, 1 second * 2 ^ retry_count)
```

Requirement FR-04: reconnect to the MQTT broker within 5 seconds of a drop.

Bench Verification

Unplug/replug test — connection dropped and manually restored, reconnection observed comfortably inside the 5-second FR-04 target.

Status: **FR-04 verified — bench unplug/replug test passed**

Notebook Verification Summary

Entry	Item Verified	Bench Result	Status
04 — RPM Measurement	Pulse-period RPM tracks live sensor pulses	Tracked correctly during bench operation	Pass
03 — Complementary Filter	Alpha selection for gyro/accel fusion	alpha = 0.98 chosen empirically	Pass
08 — PI Controller	Closed-loop step response @ 100 RPM	Settling time 2.2 s	Pass
06 — Reconnection FSM	FR-04: reconnect within 5 s of a drop	Unplug/replug recovery: passed	Pass

Note: each bench result above reflects a single representative test case. A follow-on validation pass tested these same four algorithms against a much wider range of synthetic conditions and found that two of them — the RPM stale-timeout and the PI controller's zero-setpoint behavior — do not hold up outside the specific case bench-tested here. See the companion WVTCS Algorithm Validation Report for the full analysis and fixes.